



AI Security Scanner for Python



tahirgroot@gmail.com

<https://medium.com/@tahirbalarabe2/how-to-m...>

```
## 1. SQL Injection
* **Vulnerability type:** SQL Injection
* **Why it is vulnerable:** User-supplied 'username' and 'password' are directly concatenated into the SQL query string.
* **Impact:** An attacker can manipulate the query to bypass authentication, access, modify, or delete arbitrary data in the database.
* **Secure code fix:** Use parameterized queries to separate SQL logic from user input.

'''python
def authenticate(username, password):
    conn = sqlite3.connect('users.db')
    cursor = conn.cursor()
    # Use a parameterized query
    cursor.execute("SELECT * FROM users WHERE username = ? AND password = ?", (username, password))
    return cursor.fetchone()
'''

## 2. Weak Hashing Algorithm
* **Vulnerability type:** Weak Hashing Algorithm (MD5)
* **Why it is vulnerable:** MD5 is cryptographically broken and highly susceptible to collision and rainbow table attacks.
* **Impact:** An attacker who obtains the hashed password can easily recover the original password.
* **Secure code fix:** Use a modern, strong, and slow hashing algorithm designed for passwords, such as 'bcrypt', 'scrypt', or 'Argon2'.

'''python
import bcrypt

def hash_password(password):
    # Generate a salt and hash the password using bcrypt
    hashed_bytes = bcrypt.hashpw(password.encode('utf-8'), bcrypt.gensalt())
    return hashed_bytes.decode('utf-8') # Store as string

# To verify:
# bcrypt.checkpw(password_attempt.encode('utf-8'), stored_hash.encode('utf-8'))
'''

## 3. Command Injection
* **Vulnerability type:** Command Injection
* **Why it is vulnerable:** User-supplied 'user_input' is directly embedded into a shell command executed via 'os.system()'.
* **Impact:** An attacker can execute arbitrary operating system commands on the server.
* **Secure code fix:** Avoid using 'os.system()' with user input; if shell execution is necessary, use 'subprocess.run' with a list of arguments and avoid 'shell=True'.

'''python
import subprocess

def process_input(user_input):
    # Safely execute 'echo' without shell interpretation of user_input
    try:
        subprocess.run(["echo", user_input], check=True, text=True)
    except subprocess.CalledProcessError as e:
        print(f"Command failed: {e}")
'''

## 4. Hardcoded Credentials
* **Vulnerability type:** Hardcoded Credentials
* **Why it is vulnerable:** Sensitive information like database passwords and API keys are directly embedded in the source code.
* **Impact:** If the source code is compromised or accidentally exposed, these credentials become immediately available to attackers, leading to further system compromise.
* **Secure code fix:** Store credentials securely using environment variables, a secrets management service, or a dedicated configuration file with restricted access.

'''python
import os

# Instead of: DB_PASSWORD = "password123"
# Get from environment variables
```



Introducing Today's Project!

In this project, I'm going to build a command-line tool that uses Google's Gemini AI to automatically scan Python scripts for security problems. This will help me learn how to integrate a large language model into a practical developer tool, manage API keys safely, and craft effective security prompts. I'm interested in this because manually spotting vulnerabilities like SQL injection or command injection is tedious and error prone. Automating it with AI feels like having a smart assistant that catches my mistakes before they become real bugs, plus I get to practice building something useful for my own coding workflow.

Key tools and concepts

The key tools I used include Google Gemini API for AI powered code analysis, python-dotenv for secure environment variable management, and Colorama for colored terminal output. I also used virtual environments to isolate dependencies and the command line to run my scanner. Key concepts I learnt include prompt engineering, where a structured prompt with severity levels and fixed labels gives consistent and parseable results. I also learned how to detect real world vulnerabilities like SQL injection, command injection, weak hashing, and hardcoded secrets. Finally I learned why storing API keys in .env files is critical and how color coding helps prioritize fixes.

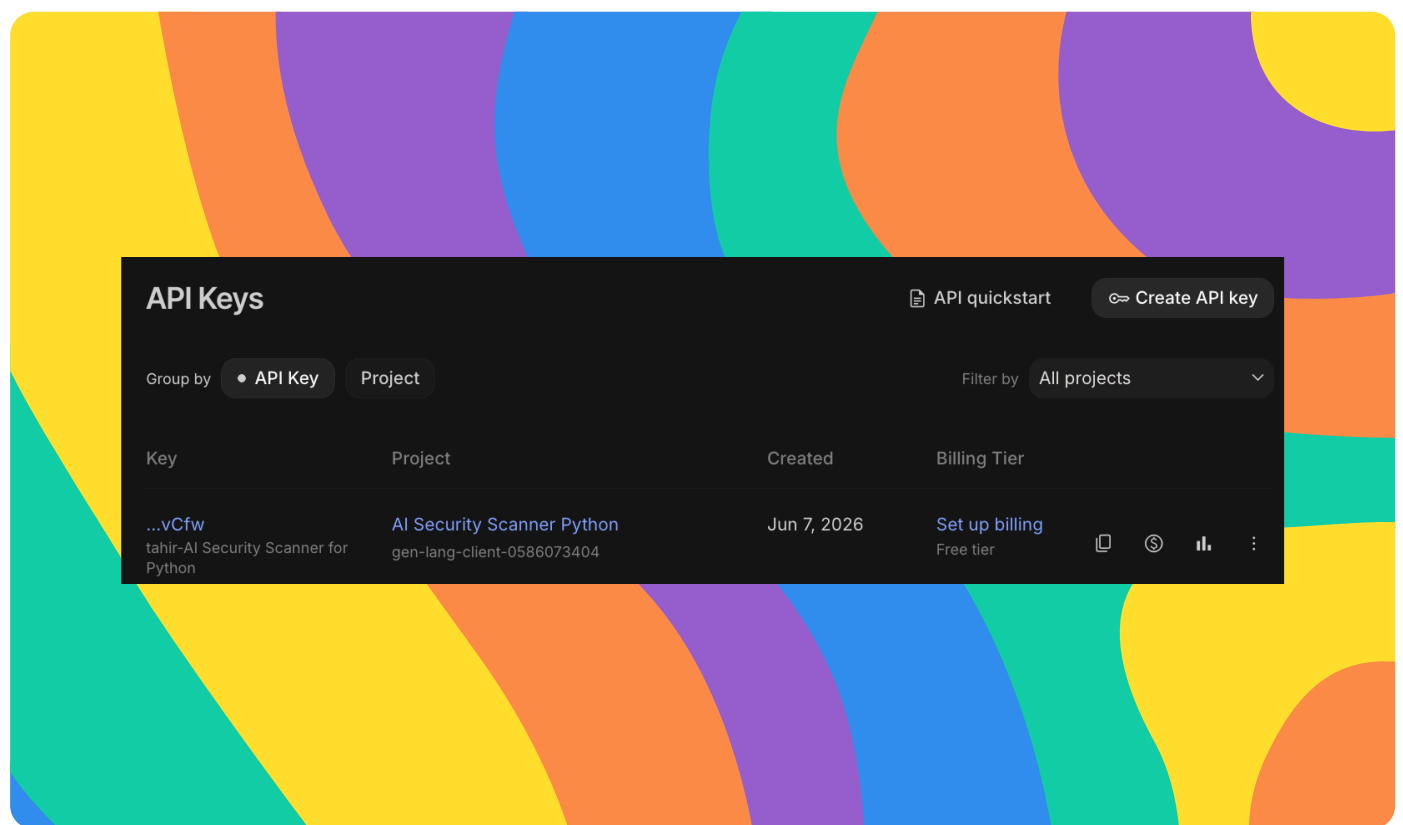
Challenges and wins

I did this project today to learn how to integrate a large language model like Gemini into a practical developer tool, specifically for automated security scanning. I wanted to understand prompt engineering, secure API key management, and how to transform an AI response into actionable, color coded output. Another skill I want to learn is how to extend this scanner to analyze entire codebases, not just single files, and maybe add automatic fix suggestions using Gemini as well. I also want to learn how to containerize this tool with Docker so I can run it in CI/CD pipelines and catch vulnerabilities before code ever gets merged.



Generating the Gemini API Key

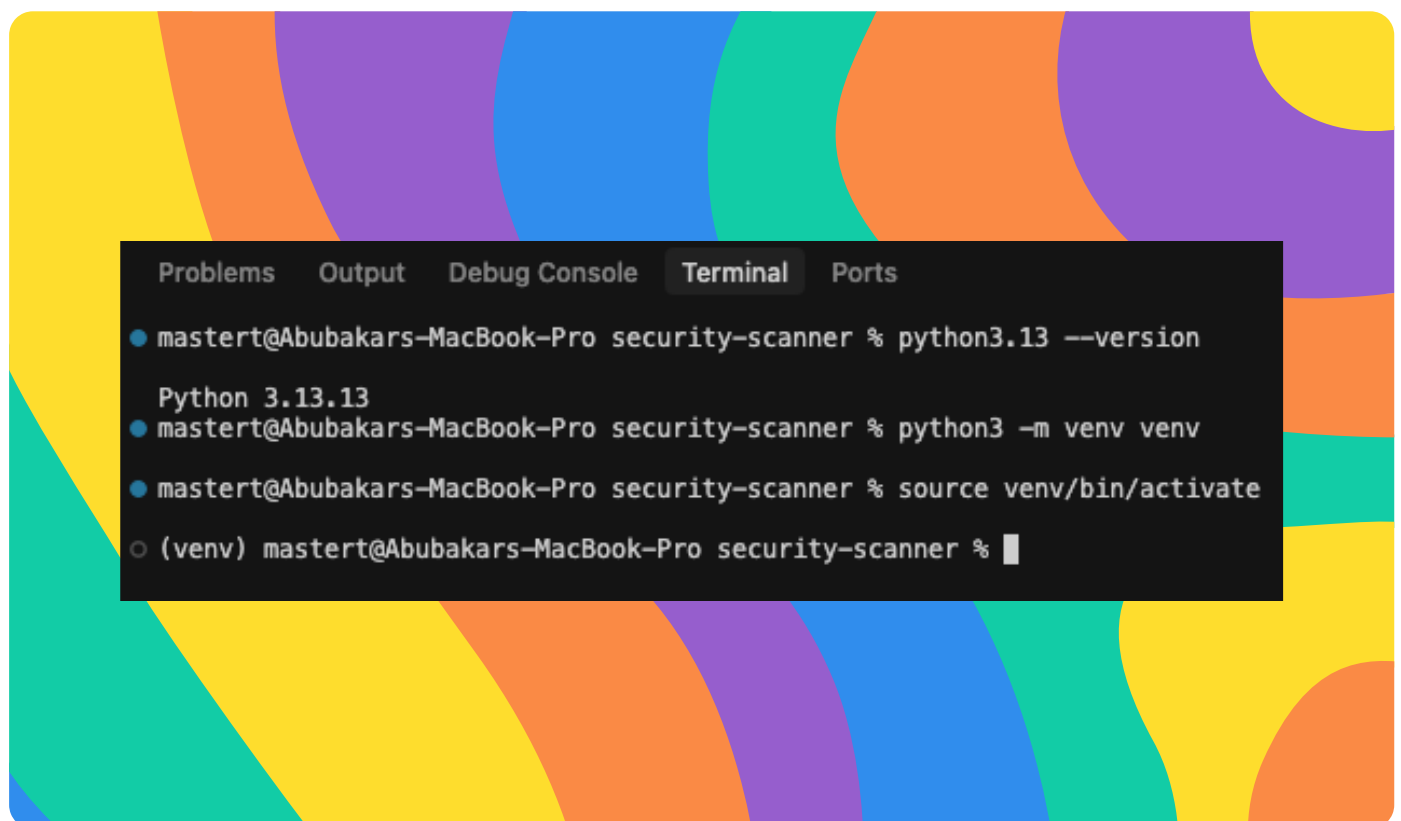
In this step, I'm setting up access to Google's Gemini AI by generating an API key. I need to do this so I can send Python code to Gemini and receive security analysis in return. Without a valid key, my CLI tool cannot call the model at all. I will visit Google AI Studio, click "Get API key," and copy the generated string. Then I will store it securely in a .env file to keep it out of my source code. This is a small but essential step before I can write the actual scanning logic or test any vulnerability detection.





Setting Up the Python Environment

In this step, I'm setting up an isolated Python environment for my security scanner project. I will create a new folder, navigate into it, and then use `python3 -m venv venv` to build a virtual environment. This is important for this project because it keeps my Gemini API dependencies separate from other Python projects, avoiding version conflicts. Activating the environment ensures that when I install `google-generativeai` and `python-dotenv`, they won't interfere with system-wide packages. A clean workspace makes my CLI tool reliable and easier to share or deploy later without unexpected library issues.



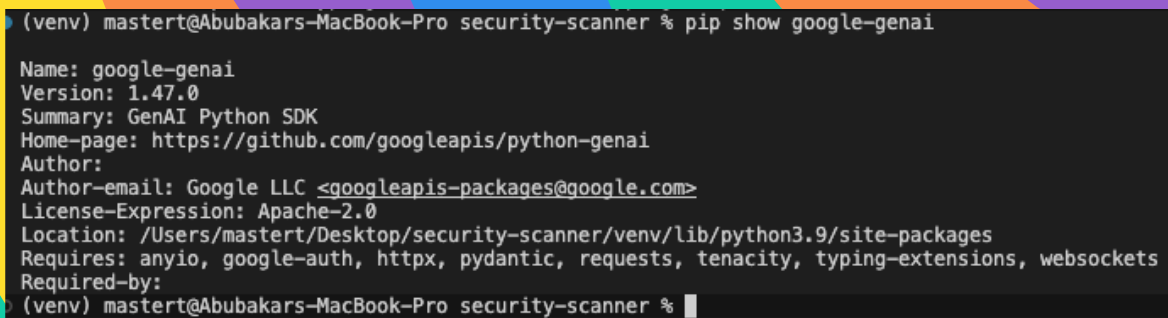
```
Problems  Output  Debug Console  Terminal  Ports
● mastert@Abubakars-MacBook-Pro security-scanner % python3.13 --version
Python 3.13.13
● mastert@Abubakars-MacBook-Pro security-scanner % python3 -m venv venv
● mastert@Abubakars-MacBook-Pro security-scanner % source venv/bin/activate
○ (venv) mastert@Abubakars-MacBook-Pro security-scanner %
```

Installing the required packages

Now that I have my virtual environment set up I need to install two essential packages: `python-dotenv` to load my Gemini API key from a `.env` file, and `google-genai` to actually call the Gemini model from Python. This is important for my project because without `dotenv` I would have to hardcode the key, which is insecure. Without the Google GenAI library, I cannot send code to



Running `pip install python-dotenv google-genai` inside my activated virtual environment gives me clean, isolated dependencies for the scanner.

A terminal window with a black background and white text. The prompt is `(venv) mastert@Abubakars-MacBook-Pro security-scanner %`. The command entered is `pip show google-genai`. The output is as follows:

```
(venv) mastert@Abubakars-MacBook-Pro security-scanner % pip show google-genai
Name: google-genai
Version: 1.47.0
Summary: GenAI Python SDK
Home-page: https://github.com/googleapis/python-genai
Author:
Author-email: Google LLC <googleapis-packages@google.com>
License-Expression: Apache-2.0
Location: /Users/mastert/Desktop/security-scanner/venv/lib/python3.9/site-packages
Requires: anyio, google-auth, httpx, pydantic, requests, tenacity, typing-extensions, websockets
Required-by:
(venv) mastert@Abubakars-MacBook-Pro security-scanner %
```



Connecting to the Gemini API

In this step, I'm connecting to Google's Gemini AI by writing Python code in scanner.py. I will load my API key from the .env file using dotenv, then initialize the Gemini client. Finally, I will send a simple test prompt like "Hello" to verify the connection works. This is important because if the connection fails, my entire security scanner cannot function. A successful test confirms that my key is valid, my environment is correct, and I am ready to build the vulnerability detection logic on top of a working API link. No scanner works without this step.



Securing the API key with a .env file

I got an error because my code tried to load the API key from an environment variable that didn't exist yet. Having a .env file is important because it keeps my secret key out of the source code. If I hardcoded the key into scanner.py and later pushed the project to GitHub, anyone could steal it and use my Gemini quota or even incur charges on my account. The .env file lets me load the key securely with python-dotenv. and adding .env to .gitignore ensures it never gets

```
(venv) master@Abubakars-MacBook-Pro security-scanner % python3 scanner.py
/Users/master/Desktop/security-scanner/venv/lib/python3.9/site-packages/google/auth/_init_.py:54: FutureWarning: You are using a Python version 3.9 past its end of life. Google will update google-auth with critical bug fixes on a best-effort basis, but not with any other fixes or features. Please upgrade your Python version, and then update google-auth.
warnings.warn(message, FutureWarning)
/Users/master/Desktop/security-scanner/venv/lib/python3.9/site-packages/urllib3/_init_.py:35: NotOpenSSLWarning: urllib3 v2 only supports OpenSSL 1.1.1+, currently the 'ssl' module is compiled with 'LibreSSL 2.8.3'. See: https://github.com/urllib3/urllib3/issues/3020
warnings.warn()
/Users/master/Desktop/security-scanner/venv/lib/python3.9/site-packages/google/oauth2/_init_.py:40: FutureWarning: You are using a Python version 3.9 past its end of life. Google will update google-auth with critical bug fixes on a best-effort basis, but not with any other fixes or features. Please upgrade your Python version, and then update google-auth.
warnings.warn(message, FutureWarning)
sdk_http_response=HTTPResponse(
  headers=dict(len=12)
) candidates=[Candidate(
  content=Content(
    parts=[
      Part(
        text=""The sky is blue primarily due to a phenomenon called Rayleigh scattering. Here's a breakdown of how it works:
1. Sunlight is White Light: Sunlight, which appears white to us, is actually a combination of all the colors of the rainbow (red, orange, yellow, green, blue, indigo, violet).
2. Wavelengths of Light: Each color of light has a different wavelength.
   * Blue and violet light have shorter, smaller wavelengths.
   * Red and orange light have longer, larger wavelengths.
3. Earth's Atmosphere: Earth's atmosphere is made up of tiny gas molecules, primarily nitrogen (about 78%) and oxygen (about 21%), along with other trace gases and particles.
4. Rayleigh Scattering:
   * When sunlight enters Earth's atmosphere, it interacts with these tiny gas molecules.
   * Rayleigh scattering refers to the scattering of electromagnetic radiation (like light) by particles of a much smaller wavelength.
   * Because blue and violet light have the shortest wavelengths, they are scattered much more efficiently and intensely by the tiny nitrogen and oxygen molecules in the atmosphere than the longer wavelengths of red, orange, and yellow light.
5. Why We See Blue:
   * As sunlight travels through the atmosphere, the blue and violet light gets scattered in all directions, bouncing around from one molecule to another.
   * This scattered blue light reaches our eyes from all angles, making the entire sky appear blue.
   * The longer wavelengths (red, orange, yellow) are not scattered as much and tend to travel more directly through the atmosphere, which is why the sun itself appears yellowish or reddish, especially when it's lower in the sky.
6. Why Not Violet?:
   * Violet light actually scatters even more than blue light. However, our eyes are more sensitive to blue light than to violet light.
   * Additionally, the sun doesn't emit as much violet light as it does blue light, and some of the violet light is also absorbed in the upper atmosphere. So, the combination makes the sky appear predominantly blue to our eyes.

In summary: The tiny molecules in Earth's atmosphere are the perfect size to scatter the shorter, blue wavelengths of sunlight much more effectively than the longer, redder wavelengths. This scattered blue light paints the sky blue for us.""
),
],
),
role='model'
),
finish_reason=FinishReason.STOP, 'STOP',
index=0
)) create_time=None model_version='gemini-2.5-flash' prompt_feedback=None response_id='7nglaq23MO_NJMKPpHfKQ' usage_metadata=GenerateContentResponseUsageMetadata(
  candidates_token_count=523,
  prompt_token_count=7,
  prompt_tokens_details=ModalityTokenCount(
    modality=Modality.TEXT, 'TEXT',
    token_count=7
  )
)
```

Verifying the connection

I verified the connection by running `python3 scanner.py` in my terminal. Gemini responded with an explanation of Rayleigh scattering, describing how shorter blue wavelengths of sunlight scatter more than red ones when interacting with Earth's atmosphere. That confirmed that my API key is loading correctly from the `.env` file, the `google-genai` library is communicating with Google's servers, and my script can send prompts and receive responses. Without that meaningful reply to "Why is the sky blue?" I would know something is wrong with the setup. Now I am ready to replace the test prompt with real security scanning logic.



Engineering the Security Prompt

In this step, I'm writing a security prompt that tells Gemini to act as a code security expert. The prompt will instruct the model to analyze Python code for SQL injection, weak hashing (like MD5 or SHA1), command injection (such as unsanitized `os.system` calls), and hardcoded secrets. It will also ask Gemini to return findings with line numbers and severity levels. This is important because without a precise prompt, Gemini might give general advice or miss real vulnerabilities. A well crafted system instruction transforms a general purpose AI into a focused security scanner that behaves consistently every time I run it.





Building the File Scanner

In this step, I'm adding file scanning so my scanner can read actual Python files from my computer, extract their source code, and send that code to Gemini for vulnerability analysis. This is important because a security scanner that only asks "why is the sky blue" is useless for real projects. I need my tool to accept a file path as a command line argument, open and read the file safely, and pass the code inside my security prompt. Once this works, I can test it on real scripts and finally see Gemini flagging actual security issues like SQL injection or unsafe system calls in my own code.

The image shows a code editor window titled 'scanner.py' with a dark background. The code is written in Python and implements a file scanner that uses the Gemini API for vulnerability analysis. The code includes imports for os, sys, dotenv, and google.generativeai. It sets up a security prompt for Gemini and reads a file from the command line argument. The code is as follows: scanner.py 1 import os 2 import sys 3 from dotenv import load_dotenv 4 import google.generativeai as genai 5 6 load_dotenv() 7 api_key = os.getenv("GOOGLE_API_KEY") 8 client = genai.Client(api_key=api_key) 9 10 security_prompt = """ 11 You are a security expert. Analyze this code for vulnerabilities. 12 13 For each issue, provide: 14 1. Vulnerability type 15 2. Why it is vulnerable (1 sentence) 16 3. Impact (1 sentence) 17 4. Secure code fix 18 19 Be concise. 20 21 Code: 22 {code} 23 """ 24 25 # File path from command line: python scanner.py <file_path> 26 if len(sys.argv) < 2: 27 print("Usage: python scanner.py <file_path>") 28 sys.exit(1) 29 30 code_path = sys.argv[1] 31 with open(code_path, "r") as f: 32 code = f.read() 33 prompt = security_prompt.format(code=code) 34 35 # Test with a simple prompt 36 try: 37 response = client.models.generate_content(38 model='gemini-2.5-flash', contents=prompt 39) 40 print(response.text) 41 except Exception as e: 42 print(f"❌ Connection failed: {e}")



Running the Security Audit

In this step, I'm testing my scanner by creating a sample Python file that contains a real vulnerability, like a SQL injection risk using unsanitized user input inside an f-string query. I'll start with one vulnerability first because it helps me verify that Gemini actually detects it and returns a clear report. If the scanner flags that single issue correctly, I can trust it for more complex cases. After that, I will add weak MD5 password hashing and a command injection via `os.system()` to see if the scanner catches all three. This step proves whether my prompt and connection work for real security tasks.



Analyzing the vulnerability report

My scanner detected SQL injection in the authenticate function, weak MD5 hashing in `hash_password`, command injection via an f-string in `process_input`, and a hardcoded database password. The most surprising vulnerability was the command injection caused by a single letter “f” inside `f"echo {user_input}"`. I never realized that using an f-string with unsanitized input could let an attacker



tahirgroot@gmail.com

NextWork Student

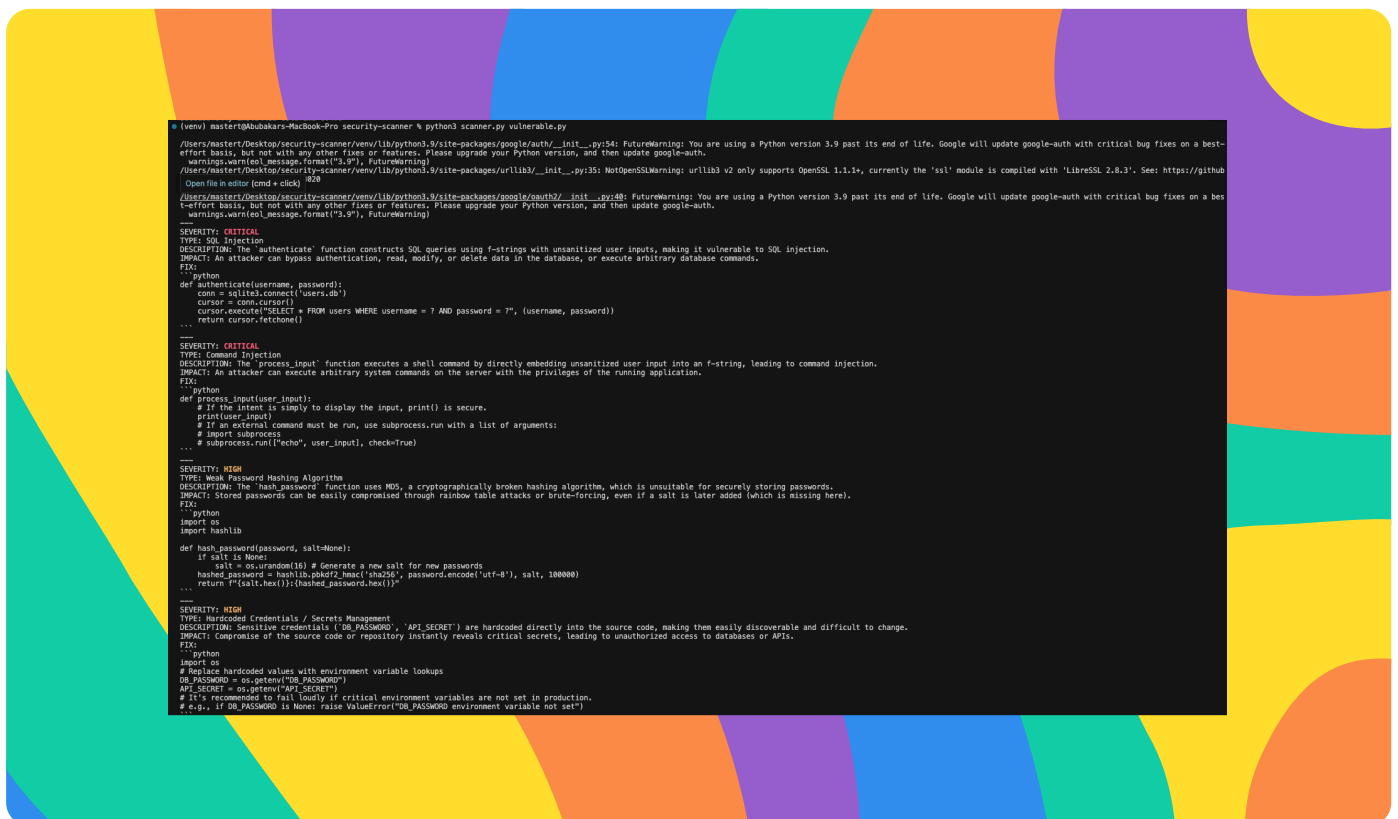
nextwork.org

This shows that AI can help with security by catching subtle flaws that look innocent at first glance, especially ones that mix string formatting with system calls. It transforms vague warnings into concrete, line-specific feedback.



Adding Color-Coded Severity Ratings

In this project extension, I'm adding color coded severity ratings so critical vulnerabilities stand out from minor issues. First, I install colorama with pip install colorama and initialize it in my script. Then I update the Gemini prompt to ask for severity levels: "HIGH" for command injection or SQL injection, "MEDIUM" for weak hashing, "LOW" for hardcoded secrets. After receiving Gemini's response, I write a small function that scans each line of the output and prints HIGH in red, MEDIUM in yellow, LOW in blue using colorama.Fore. This lets me glance at the terminal and know immediately which bug to fix first. No more guessing.



How the color system works

I added colour to my responses by first installing the colorama library and importing init, Fore, Style into my script. Then I rewrote the security prompt to ask Gemini for severity labels like CRITICAL, HIGH, MEDIUM, and LOW in a fixed format. Next, I wrote a function called add_colors to output that replaces each



tahirgroot@gmail.com

NextWork Student

nextwork.org

Now every scan shows red alerts for dangerous bugs and green for low priority issues, helping me fix the worst problems first.



Wrapping Up

This project took me approximately 75 minutes from start to finish, including the secret mission with color coding. The most challenging part was crafting the exact prompt format to make Gemini return consistent severity labels. At first, the model gave free text descriptions without the fixed structure I needed for color parsing. I had to refine the prompt several times, adding “use this exact format” and the separator lines before it worked reliably. The second challenge was understanding why my API key failed initially until I realized I forgot to load it from the .env file. Once both issues were fixed, the scanner ran smoothly and caught all four vulnerabilities.



nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

